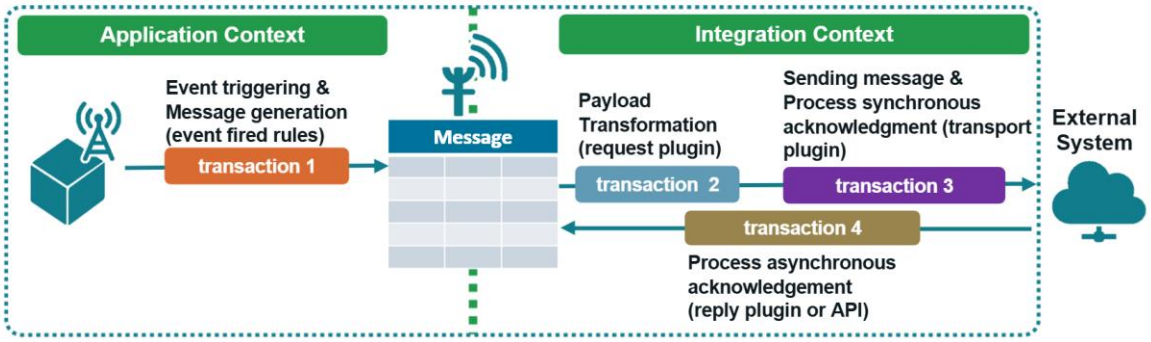
# Creating Messages

## Guidewire servers and messaging transactions




- All activity from **message generation** takes place in **transaction 1**
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

**Notes**

**Guidewire servers and messaging transactions**

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the **ui** role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the **messaging** role server.

A message is always processed in at least two and as many as four transactions:

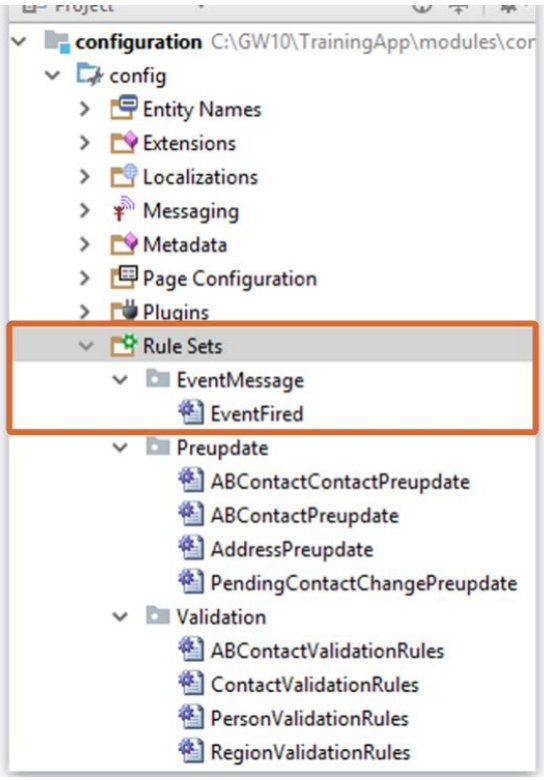
- Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
- Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
- Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
- Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction processes the acknowledgement. This transaction does not occur for messages that are acknowledged synchronously.

The transaction structure is important because this determines the work that is rolled back if an error occurs.

# Event Fired rules

## Guidewire Business Rules

- A **business rule** written in Gosu accomplishes some task for a given entity
- Three base types of rule set categories in TrainingApp
  - **EventMessage**
    - One rule set called **EventFired**
      - Root entity is MessageContext
    - Rules execute when an integration event is triggered
  - Preupdate
    - Four rule sets
    - Rules execute when an update has occurred on a root entity
  - Validation
    - Four rule sets
    - Rules check for missing information or invalid data on a root entity



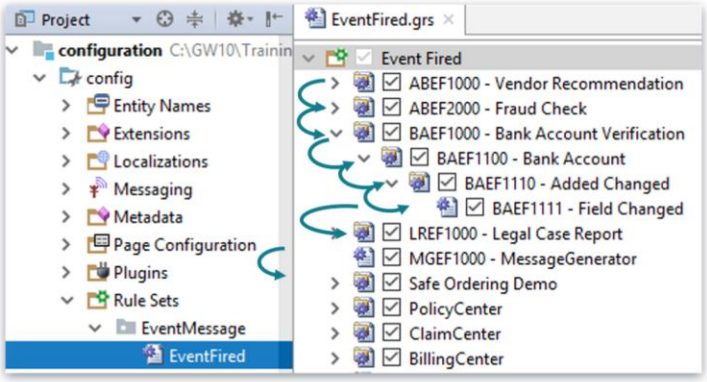

### Guidewire business rules

Business rules are covered in the Configuration Fundamentals course.

A rule set combines many individual rules into a useful set to consider as a group. A rule set is a collection of rules that share the same root entity and share common triggers. In the example above, one rule set is **Event Fired**. By selecting the rule set, the editor displays the rule set description and details of the **root entity**. In this example, the root entity is entity.MessageContext.

# Rule set execution

- Rule execution follows the order listed in Studio
- Per node, rules execute in a hierarchy
- Rule condition code resolves to true or false
  - When condition is true, rule action code executes, and
  - Child rules execute (if any)




**Notes**

### Rule set execution

In a rule set, a Guidewire application evaluates the individual rules recursively in the hierarchy, unless an exit action is encountered. The process starts with the first direct child of the root and then evaluates all children and siblings. All rules in a rule set are processed unless an explicit exit is encountered. If a rule has child rules and the parent rule condition is false, neither the parent action nor the child rules are executed.

# Rule set editor




- Rule Hierarchy
  - Context menu
  - Activate/Deactivate checkbox
  - Drag-and-drop ordering
- Rule Editor
  - Overlays segment Uses, Condition, and Action
- IMPORTANT
  - Do NOT delete the return keyword in rule condition

**Rule set editor**

You can think of a rule set as a logical grouping of rules that are specific to a business function. You typically organize these rules sets into a hierarchy that fits your business model. Guidewire strongly recommends that you implement a rule-naming scheme that is hierarchal in nature. A hierarchal naming convention will reflect the hierarchal structure when you create rules and organize rules into a hierarchy. However, the naming scheme for rules corresponds to the file structure on disk. So, if it has too many levels customers can run into problems with file names being too long on Windows.

Individual rule files have the file extension .gr. For each rule, there is a condition and action. The Rule Editor provides overlays to segment the uses, condition and action. When a rule condition evaluates to true, the Guidewire application executes the rule action and child rules, if any.

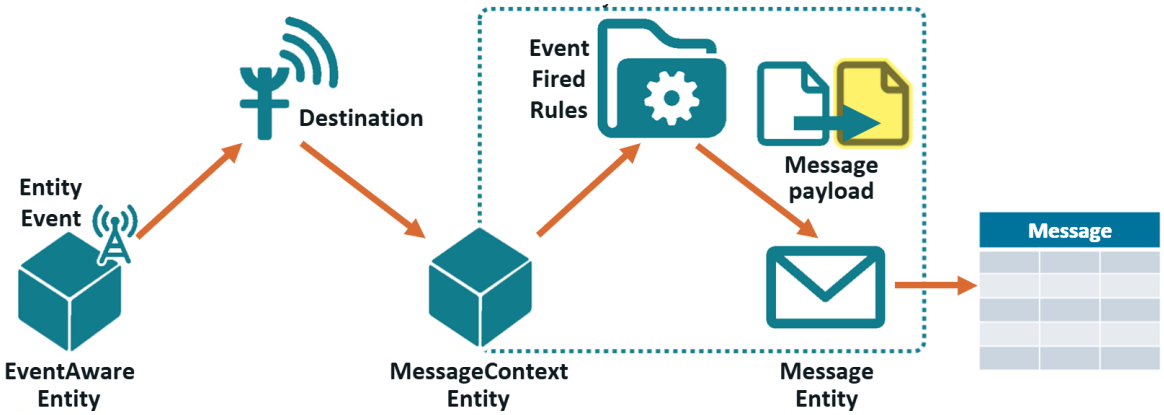
An inactive rule is not executed. You can activate and deactivate rules to get the rules engine to selective process and ignore rules. To deactivate a rule, disable the check box that appears to the left of the rule name. Inactive rules appear in gray. To reactivate the rule, enable the check box. To delete or rename a rule, right-click the rule and select the appropriate command from the context menu.

# Event Fired rule set

## Creates message and payload

### The payload typically:

- Contains static and/or dynamically generated data
- Is formatted based on the needs of the external system




**Event Fired rule set**

The application calls the Event Fired rules if an entity involved in a bundle commit triggers an event for which a message destination has registered interest. As part of the event processing, the application:

- Runs the rules in the Event Fired rule set once for every event for which a message destination has registered interest.
- Creates a MessageContext entity for every message destination of interest.
- Runs the Event Fired rule set once for each destination that is listening for that particular event. It is possible for the application to run the Event Fired rule sets multiple times for each event, once for each destination interested in that event.

For example, a system has seven destinations that listen to a given message event and it has 20 top-level Event Fired rules. Every time such an event is fired, up to 140 checks may need to be made. Thus, it is critical to keep your Event Fired rules organized to avoid unnecessary or duplicate checks.

A message payload contains static and/or dynamically generated data. For example, the payload for bank account message might contain the bank name, routing number, and account number of the given bank account as comma-separated values. The specific values for the bank name, routing number, and account number must be generated dynamically for each bank account message.

The event fired rule set is not responsible for:

- Sending messages (transport plugin)
- Acknowledging messages (transport or reply plugins)

# Common options for generating message payloads

- Gosu template
  - Formatted strings with entity data

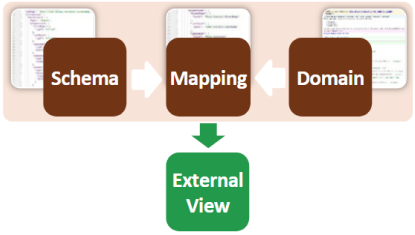





- XML modeler
  - Strongly typed XML with XSD validation
  - Supports object graph



- Integration View
  - JSON
  - Strongly typed XML with XSD validation



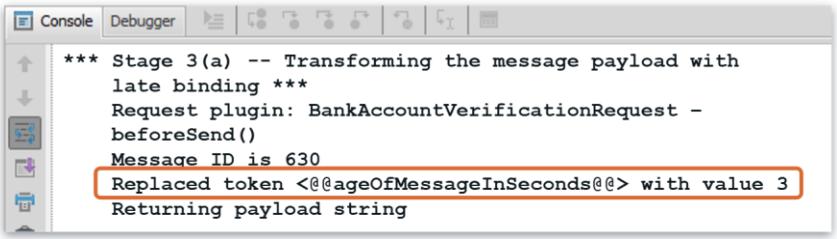

**Common options for generating message payloads**

Gosu template, XML modelers, and Integration View are not the only ways for generating a message payload, but they are commonly used. Likewise, message payload generation is not the sole purpose of a Gosu template XML modeler, or Integration View.

Other technologies that can be used to generate payloads include:

- Manually creating untyped XML using the XmlElement class
- Manually creating strongly typed XML from an XSD type
- Any logic that creates a string suitable for use as a payload

# Late binding in the payload







- A **late binding value** is a message payload value that cannot be determined until right before the message is sent
  - Use placeholders (tokens) during payload generation in the Event Fired business rules action logic (event-time)
- Message Request plugin (send-time) may search for and replace such late binding tokens with the data values immediately before the message is sent

## Late binding in the payload

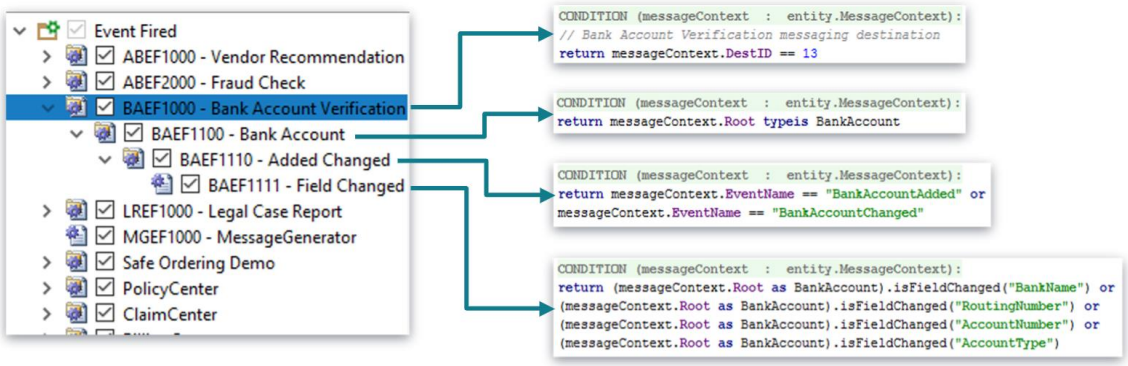
Good candidates for late binding include:

- A send timestamp
- The message ID and/or its public ID
- An identifier from the external system that was sent in reply to a previous message

For example, imagine an integration point in which the first message communicates that a new entity was added. The reply to the first message contains the ID to be used for the new entity on the external system. The second message identifies a change on this new entity. However, the second message is created before the first message has been sent (or before the reply to the first message has been received). The second message must use the ID in the reply to the first message. The only way this ID can be added to the second message is through late binding.

Typically, values that are to be late bound are added to the message payload using a placeholder marked by special character delimiters, such as <@@ and @@>. In the message request plugin, these placeholders are replaced with the required data. Be careful about what data you set via late binding. Typically, the message payload should reflect a consistent snapshot of the system at the time the Event Fired rule executed. Late binding should not be used to capture entity data after message creation.

# Event Fired rule set hierarchy structure



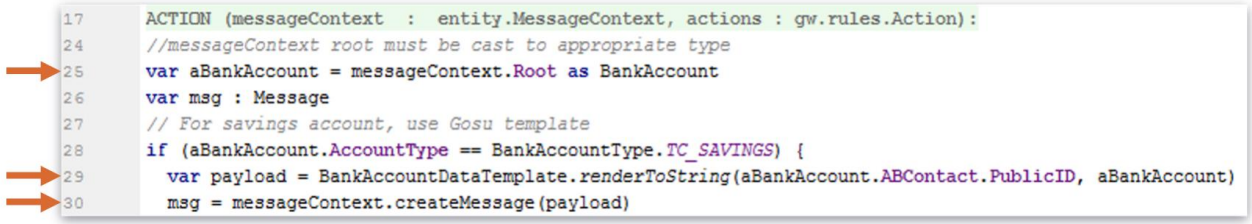

- Create a rule hierarchy to filter MessageContext objects
  - First, by destination ID...
  - Then, by root entity type...
  - Then, by event name...
  - Finally, by any other relevant logic, such as fields of interest changed

**Notes**

**Event Fired rule set hierarchy structure**

**Best practice:**
Every MessageContext is processed by the entire Event Fired rule set. However, each is typically of interest to a small subset of rules. Therefore, you should create a hierarchy to filter MessageContext objects whenever possible.

# Creating a message from MessageContext



```
17  ACTION (messageContext  :  entity.MessageContext, actions : gw.rules.Action):
24  //messageContext root must be cast to appropriate type
25  var aBankAccount = messageContext.Root as BankAccount
26  var msg : Message
27  // For savings account, use Gosu template
28  if (aBankAccount.AccountType == BankAccountType.TC_SAVINGS) {
29    var payload = BankAccountDataTemplate.renderToString(aBankAccount.ABContact.PublicID, aBankAccount)
30    msg = messageContext.createMessage(payload)
```

- Be extremely careful about modifying entity data in Event Fired rules and messaging plugin implementations
  - Entity changes in these code locations do not cause the application to run or re-run validation or preupdate rules
  - Use rules to perform only the minimal data changes necessary for integration code
    - Only change fields that are not modifiable from the user interface
    - For example, you might set custom data model extension flags only used by messaging code

**Notes**

**Creating a message from MessageContext**

**Code example**
**Line 25**
- **messageContext.Root** field is of type Object.
- To extract information from it, cast object to the appropriate type.

**Line 29**
- String payload created using a Gosu template for savings account.

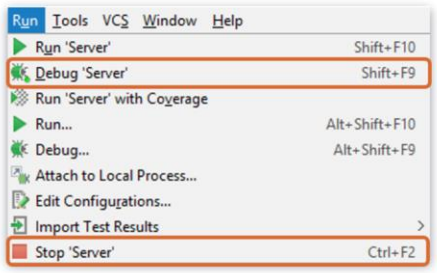
**Line 30**
- Creates the message and the rule returns a message object.

XML Modeler and Integration Views could also be used to generate XML or JSON string payloads.

## Deploy Code Changes

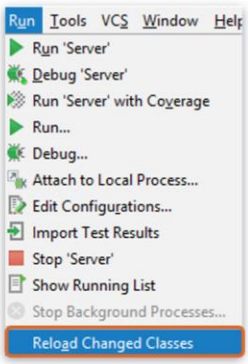
### Restart Server

- Create a rule hierarchy (parent rule and then children rules)




### Reload Changed Classes

- Newly created and modified event fired rules

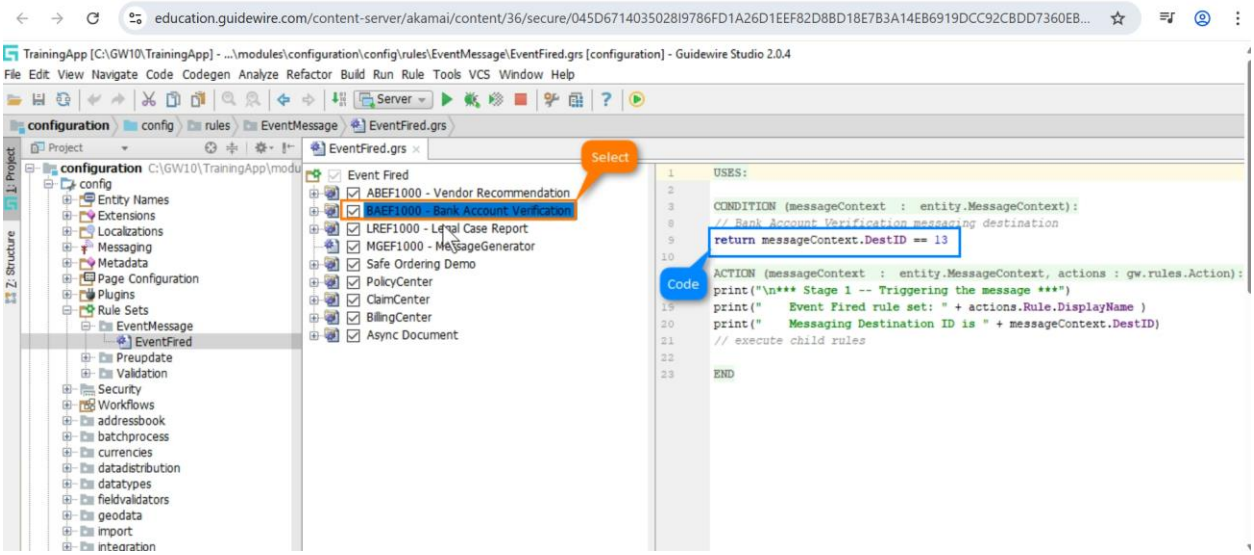


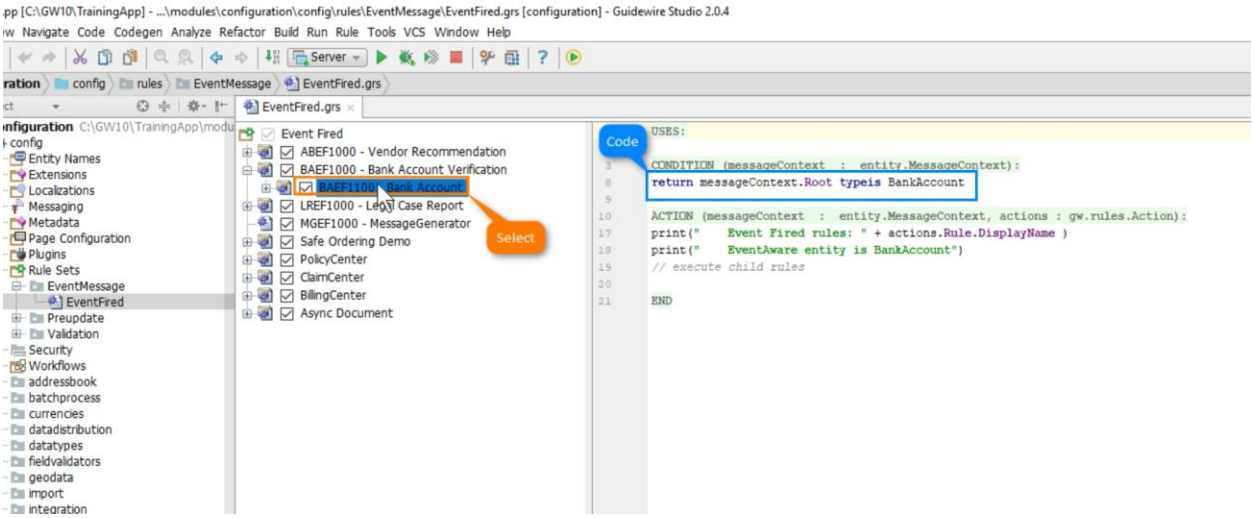

**Notes**

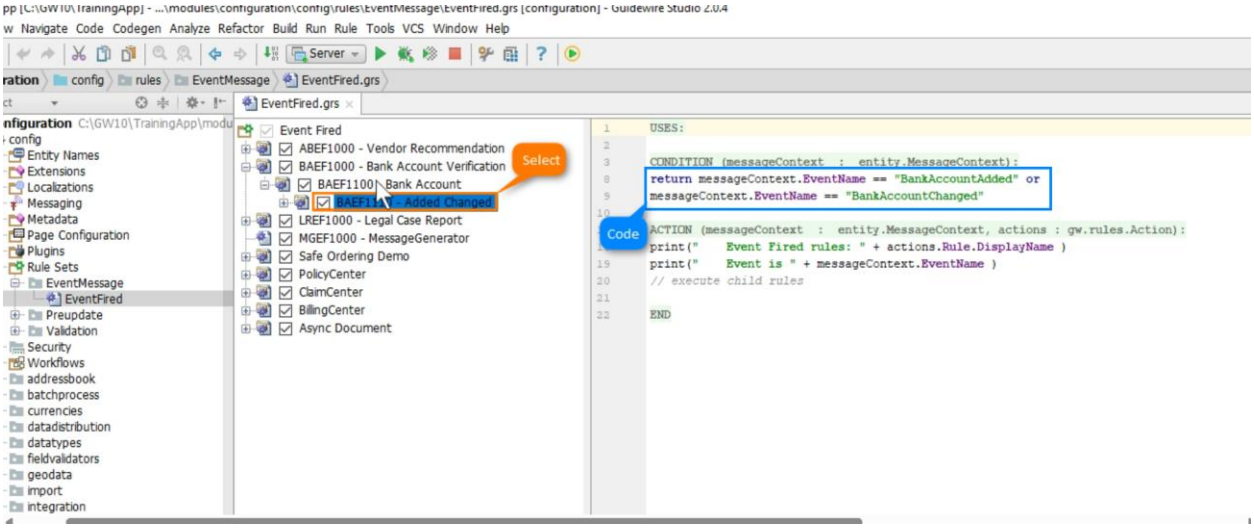
**Deploy code changes**

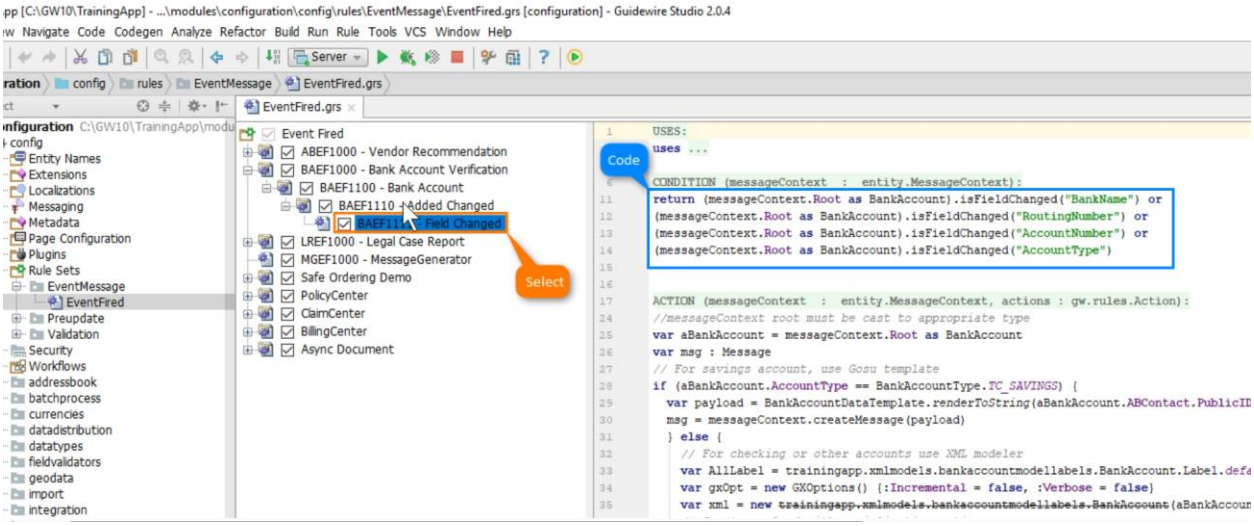
You can restart the server from Studio by stopping a running server and after the server stops, then starting the server again in either Run or Debug mode (preferred for training).

## Demonstration




Code
Select
USES:
CONDITION (messageContext : entity.MessageContext):
return messageContext.Root typeis BankAccount
ACTION (messageContext : entity.MessageContext, actions : gw.rules.Action):
print(" Event Fired rules: " + actions.Rule.DisplayName )
print(" EventAware entity is BankAccount")
// execute child rules
END
Event Fired
ABEF1000 - Vendor Recommendation
BAEF1000 - Bank Account Verification
BAEF1100 - Bank Account
LREF1000 - Legal Case Report
MGEF1000 - MessageGenerator
Safe Ordering Demo
PolicyCenter
ClaimCenter
BillingCenter
Async Document


Code
Select
USES:
CONDITION (messageContext : entity.MessageContext):
return messageContext.EventName == "BankAccountAdded" or
messageContext.EventName == "BankAccountChanged"
ACTION (messageContext : entity.MessageContext, actions : gw.rules.Action):
print(" Event Fired rules: " + actions.Rule.DisplayName )
print(" Event is " + messageContext.EventName )
// execute child rules
END
Event Fired
ABEF1000 - Vendor Recommendation
BAEF1000 - Bank Account Verification
BAEF1100 - Bank Account
LREF1000 - Legal Case Report
MGEF1000 - MessageGenerator
Safe Ordering Demo
PolicyCenter
ClaimCenter
BillingCenter
Async Document


Code
Select
USES:
uses ...
CONDITION (messageContext : entity.MessageContext):
return (messageContext.Root as BankAccount).isFieldChanged("BankName") or
(messageContext.Root as BankAccount).isFieldChanged("RoutingNumber") or
(messageContext.Root as BankAccount).isFieldChanged("AccountNumber") or
(messageContext.Root as BankAccount).isFieldChanged("AccountType")
ACTION (messageContext : entity.MessageContext, actions : gw.rules.Action):
//messageContext root must be cast to appropriate type
var aBankAccount = messageContext.Root as BankAccount
var msg : Message
// For savings account, use Gosu template
if (aBankAccount.AccountType == BankAccountType.TC_SAVINGS) {
var payload = BankAccountDataTemplate.renderToString(aBankAccount.ABContact.PublicID
msg = messageContext.createMessage(payload)
} else {
// For checking or other accounts use XML modeler
var AllLabel = trainingapp.xmlmodels.bankaccountmodellabels.BankAccount.Label.defa
var gxOpt = new GXOptions() {:Incremental = false, :Verbose = false}
var xml = new trainingapp.xmlmodels.bankaccountmodellabels.BankAccount(aBankAccoun
Event Fired
ABEF1000 - Vendor Recommendation
BAEF1000 - Bank Account Verification
BAEF1100 - Bank Account
BAEF1110 - Added Changed
LREF1000 - Legal Case Report
MGEF1000 - MessageGenerator
Safe Ordering Demo
PolicyCenter
ClaimCenter
BillingCenter
Async Document

**Question and Answer**

**Why must an Event Fired rule cast the MessageContext.Root object?**

MessageContext.Root is of type Object, but message payloads typically need to work with the MessageContext.Root as the type of entity that triggered the message event. Therefore, the MessageContext.Root entity must be cast to be of that type.